

NPU-HBM Access-Pattern Analysis for LLM Decode

How the address stream is generated, how it is spaced, and how HBM3 serves it

ONNXim + Ramulator2 · HBM3 @ 6.4 Gbps, 16 ch × 2 pch × 4 bg × 4 bank
1 KB row · 32 B request · 4 cores × 128×128 systolic, fp16
LLaMA-2 7B, one transformer block, decode · 83 simulation runs

ABSTRACT

LLM decode presents a memory system with two streams: a weight stream re-read in full on every step, and a KV cache stream whose addresses are produced by a paging allocator and therefore depend on serving history. We built a trace generator that drives a vLLM-style serving simulation into ONNXim and Ramulator2 and emits cycle-stamped DRAM addresses, and used it to characterise both streams at the interface. Decode is almost entirely a read workload: measured traffic is **99.5% reads against 0.5% writes**, since a step appends one token's keys and values per request while re-reading the whole cache.

We identify data placement, rather than request scheduling, as the primary determinant of row-buffer utilisation for LLaMA decode, and we characterise the resulting access pattern per channel. Under the layout vLLM ships, a KV read fragments into 4 KB pieces separated by a 128 KB stride, and a channel receives 4.61 accesses per activation of a 32-slot row. **We propose modifying the layout so that a head's blocks are adjacent**, which raises that to 17.16. **While-busy bandwidth goes from 45.8% to 95.4%** on the most KV-heavy synthetic workload and from 45.8% to 94.8% on the Azure trace, and end-to-end time falls by 1.22x to 2.03x across six workloads. Measured instead as a whole-run average including idle cycles, those runs read 21% against 42%, the same separation. The same row-buffer reasoning applied to the weight stream yields a second placement: laid out tile-major rather than row-major, so a tile fetch becomes one long contiguous run instead of a short strided one, it gains a comparable **22 to 23 points of row-hit rate**. Its end-to-end gain is smaller, 1.07x to 1.09x, because these runs are not weight-bound, but the stream affected is the majority of DRAM traffic over a full generation.

The result holds across grouped-query attention (1.48x), XOR bank hashing (1.24x), a 64x change in vector-unit width, and 128 requests of a production Azure trace. It collapses to exactly 1.00x under multi-query attention, as the mechanism predicts.

From a memory-system perspective, several things follow that we have not tested. Block-major placement uses only **14% of each row it opens**, which makes page policy a live question: holding rows open may be the wrong default when most of them are abandoned. The two streams also behave very differently while sharing the same banks and queues, weights running linear at 25.5 accesses per activation against KV's 4.61, so the linear stream's rows are being evicted by the strided one and per-stream bank or queue partitioning may be worth evaluating. Channel interleave granularity fixes the contiguity a stream must reach before it fills a row, and the finest interleave may not be the right trade when several cores already supply independent streams. Bank hashing recovers much of the loss while *lowering* row-hit rate, pointing at bank conflict rather than row locality as the binding constraint.

CONTENTS

PART I — TRACE GENERATION

- 1 What addresses are generated
- 2 How they are ordered
- 3 How the memory system serves them

PART II — RESULTS

- 4 Workloads
- 5 KV layout across workloads
- 6 Attention type, address mapper, and scaling
- 7 Weight layout and vector-unit sensitivity
- 8 Validation and limits

PART III — REPRODUCTION AND NEXT STEPS

- 9 Reproduction
- 10 Next: an HBM5 device model in Ramulator2

PART I

Trace generation

1 What addresses are generated

Two streams reach DRAM in decode, and their addresses are produced by different mechanisms.

The **weight stream** is static. A given layer re-reads the same 12,648,448 addresses on every decode step, because its weights do not change. Across layers the *pattern* repeats but the addresses do not: layer L issues the layer-0 list rebased by a constant stride, so it touches different memory. Weights are between 10% and 85% of a step's DRAM traffic depending on batch size and context length, dominating at small batch and giving way to KV as either grows.

The **KV stream** is dynamic. A request's cache is scattered across physical blocks chosen by a paging allocator, so its addresses depend on the admission and eviction history of every request that ran before it. This is why the generator runs a serving simulation rather than a synthetic pattern: the block table is the one component that cannot be derived from configuration.

```
workload + model + hardware + serving policy
  → serving simulation      admission, batching, block tables
  → address generation     for a chosen layout
  → cycle-stamped trace   flat, or compact whole-generation form
```

Storing a whole generation

A flat trace of a full generation is impractical to keep. Eight requests over 32 generated tokens is 452 scheduler steps across 32 layers, which is 197.7 billion addresses, roughly **4.0 TB**. Almost all of it is the same weight address list written out again once per layer per step.

Rather than compress the samples, the compact form stores **the generator and the inputs it needs**, and reconstructs any part of the trace on request. The output directory holds three files:

```
weights.u64      101 MB   one canonical weight address list
blocks.jsonl     755 KB   one record per step: each live request's id, context length, block table
manifest.json    308 B    model dimensions, layout, block size, strides, base addresses
```

Two invariances make the reconstruction exact rather than approximate. **The weight addresses repeat exactly from step to step**, since a layer's weights are fixed, and across layers the same 12,648,448-address pattern recurs rebased by $L \times 25,296,896$ B. One stored list plus that stride therefore reconstructs all 14,464 layer-step combinations. **KV addresses are a pure function** of the block table, context length, layout and block size, so the only thing that changes from step to step is the block table, a few hundred integers per step.

A window means one step and one layer, and it comes back at full fidelity in about a second:

```
$ replay_trace.py DIR --step 40 --layer 0
step 40, layer 0: 15,054,592 addresses
live requests (id, ctx): [(1,4128), (2,1836), (3,438), ...]
first 4: ['0x0', '0x90000', '0x120000', '0x1b0000']
```

That is **97.2 MB stored against 4.0 TB flat**, a factor of 42,655, with nothing lost. The window can be narrowed further to a single request or to KV traffic alone, and `--stamp` attaches arrival cycles. This is also the reason the serving simulation has to run the whole generation rather than a snapshot: the block tables are the one component that cannot be derived from configuration, because they are what an allocator produced in response to the admission and eviction history that preceded them.

What placement fixes

Layout decides which addresses are adjacent, and therefore the longest contiguous run any traversal can produce. Nothing downstream lengthens a run: the schedule can reorder but cannot re-neighbour, and the controller can only exploit locality that already exists.

The two KV layouts

The KV cache is a grid: 32 rows, one per attention head, one column per token. Memory is one-dimensional, so the grid has to be written down in some order, and the choice is normally made for the allocator's convenience.

```
      t0  t1  t2  t3
head 0 k,v k,v k,v k,v
head 1 k,v k,v k,v k,v
...
head 31 k,v k,v k,v k,v    one cell = 128 dims x 2 B x 2 (k,v) = 512 B
```

Paged attention picks token range first. The pool is cut into fixed blocks of 16 tokens and, inside a block, all 32 heads sit side by side. This is what vLLM ships and what every framework that copied paged attention inherited.

We propose the transpose: cut the pool into 32 head regions first, and index blocks inside each.

(A) **BLOCK-MAJOR**, THE DEFAULT: POOL SEGMENTED BY TOKEN RANGE



(B) **HEAD-MAJOR**, PROPOSED: POOL SEGMENTED BY HEAD



■ head 1's data [] one logical block

Figure 1 Pool segmentation under each layout. They are exact transposes. Under the default (a) the outer division is the token range, so a logical block is contiguous but head 1 is scattered into one fragment per block. Under the proposal (b) the outer division is the head, so head 1's fragments become adjacent and a logical block is slot index i claimed in all 32 regions. Both keep one block table, one free list, and 16-token granularity.

block-major (default)	$\text{off} = \text{phys} \times (32 \times 16 \times 128) + \text{head} \times (16 \times 128) + \text{tok} \times 128 + d$
head-major (proposed)	$\text{off} = \text{head} \times (S \times 16 \times 128) + \text{phys} \times (16 \times 128) + \text{tok} \times 128 + d$

only the outermost index changes

S is the pool slot count, a startup constant, so the head stride does not vary with context length and no arithmetic becomes dynamic.

Why the head is the unit that matters. Softmax normalises over tokens within a head: the denominator sums over every token of that head and no others. Head 5 cannot emit an output element until all of head 5's tokens have been read, and nothing from head 4 or head 6 enters that sum. The indivisible read is one row of the grid, and the 32 heads are independent computations that meet only at the final concatenation. That is the requirement the layout either satisfies or fragments.

block	h0	h1	h2	h3	h0	h1	h2	h3
head	h0	h0	h1	h1	h2	h2	h3	h3

Figure 2 The resulting address order, head 1 shaded. The default interleaves heads at every block boundary; the proposal concatenates a head's blocks into one long run. Weights admit the same choice: row-major storage read in tile order yields a 768 B run before a 24,576 B jump, while tile-major storage makes one tile a single 576 KB run.

What head-major costs

Writes scatter. Appending one token writes all 32 heads: one contiguous 8 KB block under the default, 32 pool-stride-apart regions under the proposal. In decode this is invisible, since measured traffic is 7,584,318 reads against 37,905 writes and writes are 0.5% of accesses. In prefill it would not be, because a whole prompt's KV is written at once. That is a real boundary on the scope of this report.

The partition is fixed at startup. Head count is baked into every address. A pool cut for 32 KV heads does not serve an 8-head GQA model without a rebuild, and cannot grow at run time without recomputing every address. The default's pool is a flat, extensible array of blocks.

It depends on the allocator. The gain requires a request's successive physical slots to be spatially clustered. ONNXim's allocator shuffles within a bounded range and vLLM's free list is a LIFO deque, so both cluster; under a uniformly random allocator the two layouts converge and the advantage disappears. Copy-on-write costs a measured 1.20×. Prefix sharing is unaffected, being a block-table operation. The benefit falls with KV head count and reaches zero under MQA, so the layout is worth nothing on an MQA model.

2 How they are ordered

Spacing is where placement becomes a DRAM cost, and the threshold is sharp.

The channel index is taken from low address bits, $\text{channel} = (\text{addr} \gg 5) \& 15$, so consecutive 32 B requests land on different channels and cycle through all sixteen. A contiguous run of L bytes therefore deposits only $L/16$ bytes in each channel. To fill a 1 KB row on every channel a run must reach

16 channels $\times$ 1024 B = **16,384 B of contiguous address**

Measure both KV layouts against it. One head's share of one 16-token block is $16 \times 128 \times 2 = 4,096$ B. Under block-major that run is bounded by the next head's data, which is not read until that head's turn, so it stays at 4 KB: **256 B per channel, a quarter row, 8 accesses per activation**. Under head-major the run is bounded by the same head's next slot, which is read immediately, so four slots concatenate to 16 KB: **1,024 B per channel, a full row, 32 accesses per activation**. Weights show the same arithmetic: a 768 B row-major run covers 48 B per channel, under 5% of a row.

Ordering contributes almost nothing

Six ordering axes exist. Two are fixed by correctness: softmax normalises over tokens within a head, which forces head-outer grouping, and dk must be innermost or the address space is not covered. Three are measurably null, including token order within a head, where all four orderings land at 96.9% because once a head is one contiguous run the order of consumption inside it changes nothing. One is live: intra-tile weight fetch order, worth 7.8 points of row hit and **0.7 points of bandwidth**, whose winner is plain sequential, which every compiler already emits.

Tiling is inert for a structural reason. The mapper pads the token dimension up to array height before tiling, so weight re-reads are pinned at exactly 1 for any batch of 128 or fewer, independent of accumulator size. Across 216 feasible tile shapes the total traffic spread is **1.03×**. Attention itself is GEMV, not GEMM: with $q_len = 1$ the score computation is $(1 \times 128) \cdot (128 \times seq_len)$, and requests cannot be batched into one matrix because each has its own context length and cache.

The visible design space is therefore small enough to enumerate exhaustively rather than search: three intra-tile orders, two layouts crossed with two allocation models, plus a non-selectable reference. Nine scored configurations, and the selection does not change across workload, attention type, address mapper or serving preset.

What the enumeration does not resolve. The analytical scorer gives block-major 87.0% row hit where the simulator measures 76 to 78%, and it scores the clustered and shuffled allocators identically, so it does not separate one of the axes it lists. Its *ranking* is validated against simulation (8 of 8); its absolute numbers are not. The allocator question was settled by simulation, not by the tool.

How the work is split across cores

Four cores share one 16-channel HBM3 system. How attention work is assigned to them changes how the four address streams interleave at the memory, though not what those streams contain.



Figure 3 Four 128x128 systolic cores against one shared memory system. Each core issues into per-channel request buffers, so a core's stream retains its own locality while the four interleave at the bank.

Table 1 The three assignment strategies (ONNXIM_PAR_STRATEGY). Only the tile-to-core mapping changes; the addresses are identical in all three.

STRATEGY	EACH CORE TAKES	CONSEQUENCE
head (default)	heads $c, c+4, c+8 \dots$ of every request	balanced; KV split, not duplicated
request	one whole request, all 32 heads	imbalanced when contexts differ
seq	one token chunk of every head	softmax denominator spans cores

Head-parallel is the default and beats the other two by 3.3x. The reason is worth stating precisely, because it is not a memory effect: the cost is **load imbalance**, with 611,200 tile rows landing on one core against 6,528 on another. Requests in a real batch have very different context lengths, so pinning a whole request to a core makes the slowest core the step time. Head-parallel divides every request the same way, so all four cores see the same amount of work regardless of the batch mix.

Two properties of head-parallel matter for the layout argument. The KV cache is **split across cores, not duplicated**: measured KV traffic scales with the number of KV heads, falling to 0.25x at GQA-4 and 0.03x at MQA, which confirms each head's data is fetched once. And because a core owns whole heads, head-major placement gives each core a long contiguous run of its own, whereas the default layout hands every core the same fragmented stride pattern.

Parallelism does not change the addresses. All three strategies issue exactly the same address set; only which core issues each tile differs. This matters for the analytical model: scoring a lockstep round-robin interleaving of four cores ranks the layouts *backwards*, while scoring one un-interleaved stream ranks them correctly, because cores issue in bursts into per-channel buffers and locality within one core's stream survives.

3 How the memory system serves them

This section characterises the arrival stream as a memory controller sees it: per channel, per bank, and in time.

One methodological point governs the rest. The raw trace is ordered by issue cycle across all sixteen channels, so a global delta histogram is a shuffle of sixteen independent streams and returns a meaningless symmetric ± 32 B distribution. The meaningful unit is **one channel's stream**. Within a channel a perfectly sequential global stream appears as a constant 512 B stride, because the intervening fifteen requests went elsewhere.

Table 2 Access-pattern classification. Channel 0, 4M-request window, runs identical but for the KV layout.

STREAM	Δ NEAR 512 B	FORWARD	ACC / ACT	P50	ROW USE	ROWS	PATTERN
weights, head-major	68.1%	91.6%	25.53	32	80%	2,638	linear
weights, block-major	68.1%	91.6%	25.53	32	80%	2,638	linear
KV, head-major	47.4%	77.1%	17.16	24	54%	284	blocked-contiguous
KV, block-major	43.6%	76.9%	4.61	6	14%	1,012	strided, 128 KB

The weight rows are a control: identical to every digit across both arms, as they must be, since KV layout does not touch weight addresses.

Weights are linear with reorder jitter. Per-channel deltas scatter around 512 B (608, 544, 480, 416: all 512 offset by multiples of 32). The addresses are sequential; the interconnect and request queue deliver them slightly out of order. The row buffer is indifferent to that, and the median row-open run is a full 32 accesses.

Block-major KV is strided, and the stride is derivable. Dominant jumps measure 128,672, -127,424, 128,576 and 127,552 B, which is 128 KB plus reorder jitter. That number is $32 \text{ heads} \times 16 \text{ tokens} \times 128 \text{ dims} \times 2 \text{ B} = 131,072 \text{ B}$, read directly off the layout formula. Every 4 KB fragment is followed by a jump over the other 31 heads. Head-major's largest recurring jump is about 1.1 KB and nothing about it is structural.

Against the prediction of \$2, 32 versus 8 accesses per activation, measurement gives 17.16 versus 4.61: the ratio of **3.72×** is confirmed, while both absolutes land near half the ideal because rows close before full consumption when 32 heads and the weight stream interleave. Block-major touches **1,012 rows against 284** for identical bytes.

Timing of arrivals

Within a burst the core issues **7.91 requests per cycle**, and the stream is close to regular: 88% of inter-arrival gaps are zero and the rest are one or two cycles, stable at 7.29 to 7.79 across the six sixths of a multi-step run. Traffic is **99.5% reads**.

Idle time is bursty and coarse-grained rather than fine. On a single core the DRAM is idle 77.5% of the run, but in only 1,182 episodes averaging 3,159 cycles: these are whole attention compute phases, not per-instruction gaps. An attention tile is 16 GEMM and 9 vector instructions over roughly 4,900 cycles with keys and values fetched up front, so the memory system is silent across the entire tile. At four cores the idle fraction falls to 4.2% because cores' compute phases interleave.

A dip during softmax does not appear, and the reason is architectural. Softmax here is fused: scores are produced into the scratchpad, normalised in place, and consumed without touching DRAM. Making it 26× more expensive (a 64× narrower vector unit with 4-cycle exp) changed idle time from 77.53% to 77.64% and the episode-length distribution not at all. In a non-fused pipeline softmax is its own kernel with its own traffic between two weight-streaming GEMVs, which produces the three-level bandwidth signature reported elsewhere. Both designs dip; only the non-fused one lets the dip be attributed to softmax.

The controller side, briefly

A ten-arm scheduling sweep on an identical address stream gave a negative result: deeper queues, a row-hit priority tier, larger active buffers and candidate retry moved while-busy bandwidth by 2 to 4 points, and two arms were bit-identical to baseline. The useful number for sizing is why. In 100% of blocked cycles no timing-ready request was present anywhere in the buffer, and the p99 reorder distance needed to repair block-major placement is **8,080 requests** against 16 for head-major. Conflict rate is a property of the arrival stream, not of the scheduler.

Two metric definitions, because they differ by more than a factor of two. *While-busy bandwidth* is transactions divided by (busy cycles x channels), counted only over cycles when a channel had work queued. The whole-run average including idle cycles is much lower and separates identically: for az128 it is 42% head-major against 21% block-major, the same 2.0x. Row-hit rate is reported as hits over *all* accesses including misses; the hits-over-(hits+conflicts) form stays at 95 to 96% under every configuration and hides the effect entirely.

Open questions for the memory system

The characterisation above is what LLaMA decode presents. What a controller should do about it we have largely not tested, so what follows are the questions our measurements raise rather than answer. The trace generator can produce whatever additional streams you would want to evaluate them against.

Channel interleave granularity. Addresses move to the next channel every 32 B, so a contiguous run is spread thinly across all sixteen: a 4 KB KV fragment deposits 256 B into each of sixteen rows and uses a quarter of every one. Switching channels less often concentrates a fragment on fewer channels but fills the rows it opens. In general a fragment of size F at granularity G leaves $\max(F/16, G)$ bytes in each row it touches, so nothing improves until G exceeds $F/16$. For the 4 KB fragment that means 256 B granularity changes nothing, 512 B gives half a row, and 1 KB gives four full rows on four channels instead of sixteen quarter-rows: row utilisation 25% to 100%, single-fragment channel parallelism sixteen down to four. Whether that trade is worth taking depends on how many independent fragments are in flight, and with four cores each working several heads there are usually enough to keep the remaining channels busy.

Address hashing. XOR bank hashing took block-major placement from 45.8% to 75.7% bandwidth while *lowering* its row-hit rate by 4 points. That is a large recovery from a cheap mechanism, and it suggests the binding constraint under fragmented placement is bank conflict rather than row locality. Whether a hash tuned against the 128 KB stride specifically does better is untested.

Page policy. Block-major placement uses 14% of each row it opens and abandons the rest. Whether rows should be held open at all under that distribution, or auto-precharged on the assumption they will not be revisited, is a decision the row-use histogram above is the right input for, and one nothing here tests. Note that prediction is not the missing piece: the core issues explicit tile fetches whose addresses the mapper computes in advance, so the stride is already known rather than needing to be learned, and fetching badly placed addresses earlier does not make them fill more of a row.

Stream separation. The two streams share banks and queues but behave nothing alike: weights run linear at 25.5 accesses per activation with a median full-row run, KV strided at 4.61. A policy that suits one is likely wrong for the other, and the mixture is what the controller actually sees. Partitioning banks or queue entries per stream, or simply tagging requests by stream, is testable against the traces already generated.

The simple bandwidth bound does not hold here. A common way to size row-buffer requirements is to ask how many accesses per activation are needed to keep the bus full. A bank cannot be reopened faster than t_{RC} , 19.1 ns on this device, and a channel has 32 banks, so activations can in principle issue at 1.68 per nanosecond. Each one delivers $acc/ACT \times 32$ B, and a channel is 51.2 GB/s:

$$acc/ACT \geq 51.2 \text{ B/ns} / (1.68 \text{ ACT/ns} \times 32 \text{ B}) = \mathbf{0.95}$$

By that reckoning row locality should hardly matter: about one access per activation would saturate the bus. Block-major placement delivers **4.61**, nearly five times the requirement, and still reaches only 45.8% bandwidth. The bound fails because it assumes all 32 banks can be activated back to back and evenly spread. In practice t_{FAW} caps activations in a rolling window, t_{RRD} spaces consecutive ones, t_{CCD_L} limits column accesses within a bank group, and refresh takes its share, so the achievable activation rate sits well below 1.68 per nanosecond. The practical consequence is that row-buffer requirements for this traffic cannot be sized from t_{RC} and bank count alone.

Read/write turnaround. Decode is 99.5% reads, so bus turnaround is close to free and none of these results exercise it. Prefill mixes in large KV writes, and head-major placement scatters exactly those writes, so turnaround cost and write placement should be examined together rather than separately.

PART II

Results

4 Workloads

Table 3 The six workloads plus a multi-step run. Context length is the KV cache resident when the step begins.

WORKLOAD	REQS	CONTEXT RANGE	MEAN	MEDIAN	STEPS	SOURCE
wl33	8	154 to 1,226	628	580	1	synthetic, short
v8m1024	8	274 to 2,600	1,023	554	1	synthetic, mean 1k
wl11	8	177 to 3,851	1,302	845	1	synthetic, wide
wl22	8	208 to 3,559	1,573	1,238	1	synthetic, long
v32m4096	32	714 to 10,025	4,095	3,608	1	synthetic, stress
az128	128	169 to 5,305	1,563	1,038	1	Azure production
azgen	8	237 to 4,089	1,170	698	32	Azure, cache grows

The synthetic four sweep KV share of traffic from 15% to 35% at a fixed batch of eight. v32m4096 raises batch and context together to reach 78%, where the effect saturates. az128 is the only point that is neither synthetic nor hand-sized. azgen runs 32 decode steps so the cache grows during the run, checking that single-step results are not an artefact of a static cache; it costs 574 million accesses.

5 KV layout across workloads

Table 4 Four cores, tile-major weights, block 16, RoBaRaCoCh. Hit rates are KV-stream specific; bandwidth is channel-level.

WORKLOAD	KV SHARE	HEAD HIT	HEAD BW	HEAD CYC	BLK HIT	BLK BW	BLK CYC	RATIO
wl33	15.4%	95.9%	98.1%	654,875	78.4%	80.6%	797,665	1.22×
v8m1024	23.2%	96.9%	98.0%	738,586	76.1%	72.1%	991,228	1.34×
wl11	28.8%	94.7%	98.4%	777,168	77.4%	69.1%	1,096,288	1.41×
wl22	34.6%	96.5%	97.5%	832,400	77.8%	66.5%	1,207,095	1.45×
v32m4096	78.2%	91.3%	95.4%	3,630,679	77.3%	45.8%	7,370,203	2.03×
az128	89.5%	94.8%	94.8%	5,652,095	76.4%	45.8%	11,176,999	1.98×
azgen	28.7%	95.6%	97.9%	24,430,212	76.0%	69.3%	34,045,646	1.39×

Block-major's row-hit rate does not move: 76.1% at one decode step and 76.0% at thirty-two, 76 to 78% at eight requests or 128, at 600-token or 4,000-token contexts. A quarter-row per channel does not improve with warm-up, longer context, or more requests to interleave. The margin tracks KV share of traffic and saturates near 2×

6 Attention type, address mapper, and scaling

Table 5 Grouped- and multi-query attention, with and without XOR bank hashing.

MODEL	KV HEADS	MAPPER	HEAD CYCLES	BLOCK CYCLES	HEAD BW	BLOCK BW	RATIO
MHA	32	RoBaRaCoCh	3,630,679	7,370,203	95.4%	45.8%	2.03×
MHA	32	XOR	3,641,817	4,532,950	94.9%	75.7%	1.24×
GQA-4	8	RoBaRaCoCh	1,380,897	2,042,142	94.2%	61.8%	1.48×
GQA-4	8	XOR	1,381,384	1,749,018	94.2%	73.2%	1.27×
MQA	1	RoBaRaCoCh	1,080,313	1,080,313	96.7%	96.7%	1.00×
MQA	1	XOR	1,079,742	1,079,742	96.7%	96.7%	1.00×

MQA is bit-identical under both mappers: with one KV head the two index orders resolve to the same addresses, so the two layouts *are* the same layout. That is the check that the mechanism is understood rather than fitted. GQA-4 retains 1.48×, which covers LLaMA-3 and Mistral. Fewer KV heads means a larger per-head slice of each block, closer to the striping threshold, so the penalty shrinks smoothly to zero.

The defensible floor is 1.24×, holding across MHA and GQA with and without bank hashing. The 2.03× headline is a best case.

Table 6 Core-count scaling at a fixed 128×128 array per core. Not iso-area: PE count scales with core count.

CORES	HEAD CYCLES	BLOCK CYCLES	HEAD BW	BLOCK BW	RATIO
1	10,799,284	11,156,132	94.4%	51.8%	1.03×
2	5,604,984	7,550,011	93.4%	45.8%	1.35×
4	3,685,039	7,364,601	93.9%	45.7%	2.00×
8	3,793,035	7,092,231	90.4%	47.3%	1.87×
16	4,023,851	4,023,851	85.5%	49.6%	1.69×

At one core the layouts nearly tie: a single core cannot generate enough outstanding requests to keep sixteen channels busy, so row efficiency is not binding. The gap opens as demand rises to meet supply. Sixteen times the compute buys 2.68× the speed and regresses past four cores, because the weight stream is fixed and extra cores add contention. PE utilisation is 3 to 4.5% at four cores.

7 Weight layout and vector-unit sensitivity

Table 7 Row-hit rate under row-major and tile-major weight storage.

WORKLOAD	ROW-MAJOR	TILE-MAJOR	GAIN	SPEEDUP
v8m1024	73.6%	95.7%	+22.1	1.08×
wl11	72.8%	95.8%	+23.0	1.08×
wl22	72.4%	95.6%	+23.2	1.07×
wl33	73.5%	95.8%	+22.3	1.09×

The speedup is modest at these batch sizes because the runs are not weight-bound, but the stream affected is 96% of traffic over a full generation. No scheduling parameter fixes it: the inner output factor is capped by accumulator capacity, and even a 512 MB accumulator reaches only a 6 KB run against the 16 KB threshold. The fix is where the bytes are written, at model load. Note that baseline ONNXim (and NeuPIMs, which embeds it) runs at the 73.6% point.

Table 8 Sensitivity to vector-unit width. Twelve runs, identical but for the vector configuration.

VECTOR UNIT	SOFTMAX	V8M1024			V32M4096		
		HEAD	BLOCK	RATIO	HEAD	BLOCK	RATIO
65536 b, exp 1 (<i>default</i>)	5 cyc	738,586	991,228	1.342×	3,630,679	7,370,203	2.030×
8192 b, exp 2	18 / 66	738,844	984,626	1.333×	3,625,550	7,442,563	2.053×
1024 b, exp 4	130 / 514	741,750	985,033	1.328×	3,671,272	7,534,081	2.052×

The default vector unit is 4,096 fp16 lanes with single-cycle exp, which makes softmax cost 5 cycles at any context up to 4k: $O(1)$ where the operation is $O(seq_len)$. That is not defensible hardware, so it was tested. Narrowing the unit 64× and quadrupling exp latency moves the ratio by at most 1%, and at 4k context it moves *up*.

8 Validation and limits

Table 9 Checks against the cycle-level simulator.

CHECK	RESULT
Analytical row hit vs simulator	±1.1 points, both streams
KV address counts	8 of 8 exact
Weight address counts	1.0002
Tiling vs simulator mapper	reproduces exactly
Layout ranking vs simulation	8 of 8
Timing model vs multi-step run	0.1% aggregate
Run-to-run variance	bit-identical
Baseline reproduced in the sensitivity re-run	exact to the cycle
Analytical <i>magnitude</i>	not validated, understates block-major by 10 points

Scope. One transformer block, one simulator, no silicon. Integer-8 and block sizes other than 16 are untested against the stride-sensitive results; int8 halves every fragment, which should widen the gap, but that is a prediction. Prefill is out of scope, and the write-scatter cost of head-major would need measuring there. Every claim is a decode claim.

On the printed peak bandwidth. The simulator's startup line reports DRAM Bandwidth 1638 GB/s, which is a display artifact rather than a configuration error. `max_dram_bandwidth()` divides by the config field `dram_nbl`, set to 1 where a 32 B request at BL8 occupies 2 command clocks; with `dram_nbl: 2` the same line prints 819.2 GB/s, which is HBM3 at 16 channels × 2 pseudochannels × 32 bit × 6.4 Gbps. The field is used nowhere else, so simulated bandwidth is governed entirely by the Ramulator timings and no result here depends on the printed value.

PART III

Reproduction and next steps

9 Reproduction

```

docker build . -t onnxim
./scripts/devshell.sh ./scripts/build_in_docker.sh          build dir must be named build/

./scripts/devshell.sh bash -c '
  cd /workspace/ONNXim/build && \
  ONNXIM_DRAM_TRACE=/workspace/ONNXim/out/run.csv ONNXIM_DRAM_TRACE_LIMIT=30000000 \
  ONNXIM_WEIGHT_SWIZZLE=1 ONNXIM_KV_LAYOUT=head ONNXIM_KV_BLOCK=16 \
  ./bin/Simulator --config ../configs/_c128.json \
  --models_list ../example/v8m1024.json --mode language --trace_file v8m1024.csv'

trace format  cycle,channel,pseudochannel,bankgroup,bank,row,column,address,rw,core
                7,0,1,0,0,192,12,0x6004c00,R,0

```

Three traps. `--trace_file` is required and overwrites the models-list JSON field, so two runs differing only in that field produce identical results. The ONNXim JSON `dram_*` fields and the Ramulator2 YAML organisation must be kept in sync by hand: the trace decoder recomputes channel, bank and row from the raw address using the JSON, so a mismatch leaves the simulation correct and every decoded field wrong. And `cycle` is in the DRAM domain at 3200 MHz, not the 1 GHz core clock; divide by 3.2 to compare against the printed `Total cycle`.

Table 10 Environment knobs. Behaviour is controlled by environment rather than config fields, so a sweep is a list of prefixes over one binary.

VARIABLE	VALUES	EFFECT
ONNXIM_KV_LAYOUT	head / block	the proposal vs the default
ONNXIM_KV_BLOCK	tokens	paging granularity
ONNXIM_KV_ALLOC	seq / shuffle	block-table allocation model
ONNXIM_WEIGHT_SWIZZLE	0 / 1	row-major vs tile-major weights
ONNXIM_INTRA_ORDER	seq / col / blk	intra-tile weight fetch order
ONNXIM_PAR_STRATEGY	head / req / seq	work split across cores
ONNXIM_DRAM_OCC	0 / 1	per-channel busy-cycle counters
RAMULATOR_REQBUF	entries	request-queue depth
RAMULATOR_SCHED_ROW_HIT	0 / 1	row-hit priority tier
RAMULATOR_STARVE_CAP	cycles	starvation bound on that tier
RAMULATOR_ACTBUF	entries	active (open-row) buffer size
RAMULATOR_ROWTRACE	path	per-activation row trace

For whole generations, `tops_visible.py --emit-compact DIR` then `replay_trace.py DIR --step N --layer L`, adding `--stamp` for arrival cycles. Analysis of a sweep directory is `analyze_sweep.py`.

10 Next: an HBM5 device model in Ramulator2

Every effect in this report is a row-buffer and channel-stripping effect, and both change with the device generation. That is the reason to build the model, and it also predicts what the model will show.

Device models live in `extern/ramulator2/src/dram/impl/`, eleven of them today. HBM3.cpp is 427 lines and is the template. A model declares five things, of which two carry the work.

Table 11 The parts of a Ramulator2 device model, with HBM3's values.

PART	HBM3	FOR HBM5
Registration macro	RAMULATOR_REGISTER_IMPLEMENTATION(IDRAM, HBM3, ...)	name only
Hierarchy <code>m_levels</code>	channel, pseudochannel, bankgroup, bank, row, column	likely unchanged
Prefetch size	<code>m_internal_prefetch_size = 2</code>	candidate change
Organisation presets	<code>{density, DQ, {Ch, Pch, Bg, Ba, Ro, Co}}</code>	main change
Timing presets	26 values, one preset	main change

```
// name    density  DQ   Ch Pch Bg Ba  Ro   Co
{"HBM3_8Gb", {8<<10, 128, {1, 2, 4, 4, 1<<15, 1<<6}}}
```

```
timing: rate nBL nCL nRCDRD nRCDWR nRP nRAS nRC nWR nRTPS nRTPL nCWL nCCDS nCCDL
        nRRDS nRRDL nWTRS nWTRL nRTW nFAW nRFC nRFCB nREFI nREFISB nREFED tCK_ps
```

Steps

1. Copy and register. HBM3.cpp to HBM5.cpp, rename the class, update the macro's three strings, add `impl/` HBM5.cpp to `extern/ramulator2/src/CMakeLists.txt` (HBM3 is line 18). Registration is self-contained, so the YAML can then say `impl: HBM5`.

2. Define the organisation. This sets the row size and therefore the striping threshold, which is the quantity every result here turns on. Row bytes are $\text{columns} \times \text{prefetch} \times (\text{DQ} / \text{pseudochannels}) / 8$, giving 1024 B in the present configuration, and the threshold is $\text{channels} \times \text{row_bytes}$.

The prediction to test first. If HBM5 doubles the channel count, the contiguity threshold doubles from 16 KB to 32 KB and *block-major placement gets worse, not better*: a 4 KB fragment would cover an eighth of a row rather than a quarter. The layout penalty is not a fixed tax; it scales with channel count. Head-major placement, whose runs are already far above the threshold, should be unaffected.

3. Build the timing preset. Ramulator2 refuses to combine a timing preset with a rate override, so a full explicit timing set is the only way to model a data rate the presets do not cover. The method already used for HBM3 at 6.4 Gbps applies: hold the absolute nanosecond timings of a known preset constant and re-express them in cycles at the new clock, $\text{ceil}(t_{\text{ns}} \times 1000 / t_{\text{CK_ps}})$. Two values must not be time-scaled because they are bus-structural rather than DRAM-core delays: `nBL`, the burst length in bus cycles, and `nCCDS`, which must cover the burst so that $\text{nCCDS} \geq \text{nBL}$.

4. Mirror the organisation into the ONNXim JSON. `dram_channels`, `dram_pseudochannels`, `dram_bankgroups`, `dram_banks`, `dram_rows`, `dram_columns`, `dram_channel_width`, `dram_prefetch_size` and `dram_freq` must all match, or the trace decoder silently reports wrong banks and rows.

5. Re-validate and re-run. The analytical model parses geometry from the JSON and YAML rather than hardcoding it, so it should follow automatically, but `--validate` and `--verify` - ranking should be re-run before any conclusion is drawn. The minimum re-run set is the six-workload comparison, the GQA and mapper ablation, and the controller arms. The last is most likely to change, since the negative controller result rests on there being no timing-ready request during blocked cycles and different timings could alter that.